

Your GPU kernel and cache, explained simply

atomOS ASA v3 | Local validation and improvements
13 September 2026 | Concept author: Tom Klootwijk

The native GPU kernels work on this laptop. The corrected program passes the numerical checks, performs real texture fetches, and now groups nearby lookups together. That reduces texture misses and roughly halves the warmed kernel time in the measured workload.

80.5%

fewer Morton texture misses

0

observed warm L2 read misses

2.05x

faster warmed Morton kernel

0

retained oracle numerical error

The useful result

Imagine a cook fetching ingredients. The old work order sends the cook back and forth. The new order groups nearby ingredients, then puts the finished answers back in their original order. The recipe and the answers stay the same.

What "resident" means here

The repeated warm runs found the requested data in the GPU's shared L2 cache. This is observed reuse during those runs. It does not establish permanent cache-line pinning, a full cache, or maximum memory bandwidth.

CPU ring 0 is a separate issue. ASA is a user-mode CUDA application using the installed NVIDIA driver. The code executes on GPU execution units; the texture cache holds lookup data. The repository contains no custom ring-0 driver. [S1, E6]

Measured on NVIDIA GeForce RTX 5070 Ti Laptop GPU, compute capability 12.0. Comparison: 65,536 identical samples, 512 threads per block. Kernel speed excludes host sorting, transfers, verification and file output. Evidence and reproduction are on page 6. [E1-E4]

Does it fill the entire cache?

No. The needed data fits in cache; filling the entire cache is a different claim. The default image is 512 KiB. The GPU reports 36 MiB of L2 capacity, which is 72 times larger. Adding unused data would not help this calculation.

Measurement	What it tells us
Cache hit rate	How often a requested read finds data at that cache level.
Cache capacity occupied	How many bytes currently sit in cache. We did not measure this.
Bandwidth saturation	Whether traffic reaches the hardware transfer limit. A high hit rate does not prove this.
GPU occupancy	How many warps can be resident. This is distinct from cached data.

Two cache levels, two different jobs

L1/TEX is close to each GPU execution unit group (SM). **L2** is shared across the GPU. A texture read can miss the smaller cache and still find its data in L2. That is why a roughly 71% TEX hit rate can coexist with zero observed warm L2 read misses. [E2]

Capacity or allocation	Size / observation
Image table	512 KiB
Matte and lens tables	16 KiB and 4 KiB
Whole counted GPU payload	3,690,496 bytes, including inputs and one reused output allocation
Shared L2 capacity	36 MiB, reported by this device
L1/TEX plus shared capacity	128 KiB per SM in the CC 12.0 architecture [S2]
Measured shared partition	8 KiB; application static/dynamic shared memory: 0; driver: 1 KiB per block [E5]

Subtracting that 8 KiB partition from the documented 128 KiB pool gives a 120 KiB L1/TEX budget per SM. This is a capacity inference, not a measurement of occupied cache bytes. The launch already uses a small shared partition; a large application shared-memory reservation is not the cause of its TEX misses. [S2, E5]

What improved, and what it costs

A stable host-side ordering groups samples by an approximate lookup location. The actual GPU kernels still compute the complete ASA operator. After download, a checked permutation restores every result to its original sample position. Natural and reordered result bytes match exactly. [E1, E3]

Layout / order	TEX hits	Missed sectors	Texture time, us	Global time, us
Linear / natural	62.89%	62,360	60.47	60.37
Linear / locality	85.27%	10,035	29.63	29.95
Morton / natural	67.96%	53,652	60.71	61.21
Morton / locality	71.21%	10,470	29.65	29.37

These are medians from the final binary. Timing: five separate runs per condition, each with five warmups and 30 timed launches per path. Profiling: three cold-cache captures per condition, base clock control. The TEX metric counts texture-load sectors specifically. [E1]

Why the percentage is only part of the result

Morton locality reduced requested texture sectors from **167,449 to 36,366**, and misses from **53,652 to 10,470**. The TEX hit rate rose by 3.25 percentage points, but misses fell by 80.5%. Linear achieved a higher hit percentage while requesting more sectors. Neither percentage alone tells the whole performance story.

Host work is a real cost

For Morton locality, sorting took a median **2.335 ms**; restoring the texture and global outputs together took **2.071 ms**. These costs exceed the roughly 0.031 ms saved by one texture launch. Repeated launches amortize preparation; a one-shot caller must account for it.

Entire benchmark-invocation wall-time medians were 1,496 / 406 ms for linear natural/locality and 1,505 / 1,570 ms for Morton. These include 70 kernel launches, startup, CPU checks, transfers and file output, and varied substantially. No general end-to-end speedup is claimed. [E1]

Selected GPU defaults: locality ordering and 512 threads per block. The explored 1024-thread setting raised Morton TEX hits to about 75%, but slowed the warmed kernel to about 44 us. It was not selected. L2 policy remains normal. [E1, E7]

Native cache reuse: what was proven

The residency probe ran **84 captures**: cold/warm caches, normal/persist-image policy, texture/global kernels, seven metric groups, three repetitions. Each accepted capture used exactly one profiler pass and preserved the original sample and result bytes. [E2]

Texture kernel counter	Cold, normal	Warm, normal
L2 read miss sectors	39,189	0 in all 3 captures
L2 read hit sectors	4,049 median	43,234 median
TEX hit rate	71.21% median	71.22% median

L2 counters cover read traffic from the shared L1/TEX source unit, not just individual image lines. Hits and misses were captured separately, so this report does not combine them into a same-launch L2 hit percentage. Output writes are outside this read-miss statement. [E2]

The optional retention hint

--l2-policy persist-image requests preference for the 512 KiB image. CUDA accepted a 512 KiB address window and rounded the requested 512 KiB set-aside to **2.25 MiB**. The stream policy was read back successfully. Its hitRatio of 1 is a policy parameter, not a measured 100% cache hit rate. [E2, S3]

Observed evict-last read sectors	Normal	Persist-image
Texture kernel, cold and warm	0	0
Global-load control, cold median	0	8,230
Global-load control, warm median	0	8,225

The control shows that the hint reaches ordinary global loads. No retention-priority propagation was observed for the texture path on this device/build, and the hint did not lower its already-zero warm L2 read misses. It therefore remains optional; it is not presented as a texture pinning fix.

Accurate conclusion: native texture execution and warm L2 reuse are demonstrated. Permanent texture residency, cache occupancy and behavior under competing cache pressure remain unproven.

An initial probe requested multiple L2 counters together and needed four replay passes. It was rejected. Separate one-pass captures avoid claiming a warm-cache result from replay-altered cache state. [E2, S4]

Correctness checked before delivery

Executed check	Result
CPU and GPU builds	CPU: 3 CTest targets passed. GPU: 5 passed; native sm_120 plus PTX.
C++ and Python regressions	33 C++ groups / 74,982 assertions; 42 Python tests passed.
CLI integration	58 CPU and 76 GPU scenarios, including zero-size and clean failure checks.
Saved numerical records	65,536 samples per layout plus 72 boundary cases / 28,656 samples. Independent maximum error: 0.
Stream lifetime regressions	Normal exit, exception unwind and explicit close; 65,536 samples each compared with CPU.
Compute Sanitizer	16 invocations across memcheck, initcheck, racecheck and synccheck. Zero reported errors; racecheck also zero hazards/warnings.

Defects repaired

Zero-input metadata now reports that no GPU kernel ran. Windows validation launches the native sanitizer executable and records unavailable tools honestly. Python checks are mandatory, rejected CLI inputs must exit cleanly, and the CUDA build uses a compatible static MSVC runtime. Stream cleanup waits before textures can be released. Each correction has regression coverage. [E3, E4]

The numerical contract stays intact

Pair formation still precedes aperture gating, then lens warping and table lookup. Flags and logical cell keys agree exactly; float tolerance remains **2e-6**. The core arithmetic and independent Python reference are unchanged. Explicit Morton8 storage stays separate from NVIDIA's internal cache layout. All 30 bundled result files retain their original hashes. [E3, E6]

Limits: these synthetic checks are not physical lens calibration or proof for every possible input. Low-VRAM admission boundaries use simulated free-byte values. Minimum/maximum LUT-size tests use identity coefficients; nonzero warp is checked at 512 bins. Current Windows host ASan/UBSan is unavailable (exit 2), not passed. Racecheck covers shared-memory hazards, not every possible global-memory race. [E3, E4, S5]

Reproduce it and inspect the evidence

From the repository root in PowerShell, use a fresh output directory for every run. README contains the full toolchain, benchmark and profiler commands. [E6]

```
python scripts/validate.py --gpu --sanitizer `
--build-dir b128 --report-dir local_validation/my_gpu_run `
'--cmake-arg=-Tcuda=C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.8'
```

PowerShell backticks continue a command onto the next line. Run the selected native path:

```
.\b128\Release\asa_cuda.exe --layout morton `
--sample-order locality --block-size 512 `
--warmup 5 --repeat 30 --out new_gpu_run
python scripts/verify_run.py new_gpu_run
```

Optional experiment: add **--l2-policy persist-image**. The readback describes the configured hint; use the profiler probe to measure its effect. Kernels reuse unchanged device tables during repeated launches. Cleanup clears the window and restores the previous limit; resetting persistence is context-wide, so this ownership model is for the standalone CLI. [E6]

Local evidence index

E1-E5 and E7 are beneath [local_validation/20260913_kernel_review/](#).

ID	Record
E1	cache_final_512/benchmark.json
E2	residency_final/residency_probe.json
E3	final_records/records_evidence.json; core_details.log and python_details.log in that same directory
E4	final_gpu/status.json; final_cpu/status.json; host_sanitizer_unavailable/status.json
E5	l1_capacity.log
E6	Repository: CODEX_FINDINGS.md, README.md, docs/PRIVILEGE_CONTRACT.md
E7	block_exploration.json; source_changes.patch; source_snapshot.json

Primary technical references

- S1 [Microsoft: user mode and kernel mode](#)
- S2 [NVIDIA CUDA 12.8: compute capability 12.0](#)
- S3 [NVIDIA CUDA 12.8.1: L2 access management](#)
- S4 [NVIDIA Nsight Compute: replay and cache control](#)
- S5 [NVIDIA Compute Sanitizer: tool coverage](#)

Environment: Windows 11 Home 26200; driver 591.59; CUDA driver API 13.1 / runtime 12.8; nvcc 12.8.61; MSVC 19.44.35221; Compute Sanitizer and Nsight Compute 2025.1. The original delivered PDF and package manifest were not replaced.